

CISCO VIRTUAL INTERNSHIP 2026

Internship Evaluation Presentation

SmartBranch 360 — Secure Branch-Office Network

STUDENT

Akhil Gwari (2410030441)

PROGRAMME

B.Tech — Computer Science Engineering, IILM University

ROLE

Member 3 — Python Tool, Documentation & Demo

What This Presentation Covers

- 01** **Internship Programme** Cisco Virtual Internship 2026 and prerequisite certifications
- 02** **Project Overview** SmartBranch 360 topology and team structure
- 03** **My Contribution** Python Tool, Documentation & Demo — Member 3 responsibilities
- 04** **Assurance & Verification** Requirements spec, checker tool, fault-card documentation
- 05** **Outcomes** Skills gained and conclusion

SECTION 1

Internship Programme Overview

Delivered through the Cisco Networking Academy at IILM University, Greater Noida — combining self-paced technical training with a hands-on, team-based networking project built entirely in Cisco Packet Tracer.

3

Certifications completed

3

Team members on the project

21/21

Assurance-tool checks passed

PROGRAMME SNAPSHOT

PROGRAMME

Cisco Virtual Internship 2026

DELIVERED VIA

Cisco Networking Academy / IILM University

PROJECT

SmartBranch 360

PLATFORM

Cisco Packet Tracer

VALIDATION

Python-based configuration assurance tool

SECTION 1

Prerequisite Certifications

Three Cisco Networking Academy courses were completed ahead of the group project to build the required foundational skills.

1

Networking Essentials

COMPLETED

16 Jun 2026

2

Getting Started with Cisco Packet Tracer

COMPLETED

16 Jun 2026

3

Exploring Networking with Cisco Packet Tracer

COMPLETED

17 Jun 2026

SmartBranch 360 — Project Overview

A secure branch-office network built in Cisco Packet Tracer — supports wired employees, guest Wi-Fi, an internal server, Internet access through NAT, and secure administrative management, all validated by a Python-based assurance tool.

MEMBER 1

Core Network Build

VLANs, trunking, Router-on-a-Stick, DHCP

MEMBER 2

Security, NAT & Testing

Guest ACL, SSH hardening, NAT, end-to-end testing, fault injection

MEMBER 3

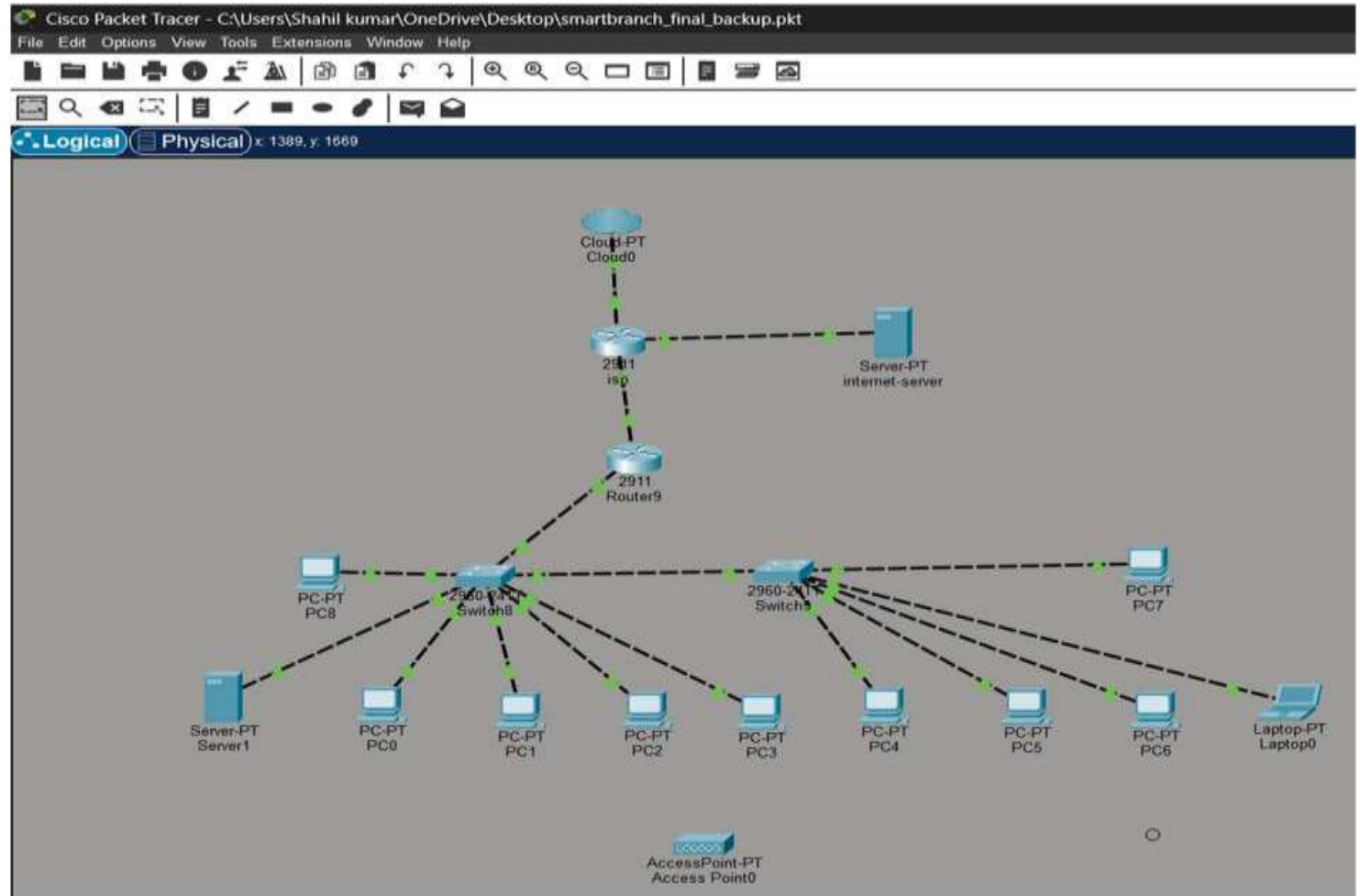
Python Tool + Documentation + Demo

Assurance tool, requirement spec, design document, fault cards, demo video → This report

Shared Network Topology

TOPOLOGY COMPONENTS

- Cloud/ISP simulation + ISP router
- R1 (Router9): Router-on-a-Stick inter-VLAN routing
- SW1 / SW2 access-layer switches
- Internal server + internet-facing server
- Wireless access point
- 8+ endpoints across 4 VLANs



SECTION 3

My Contribution — Python Tool, Documentation & Demo

As Member 3, I wrote the YAML/requirement specification, built the Python assurance tool that checks the shared configuration against it, compiled the group's design document, documented the five fault-injection cards, and recorded the demo video.

1

Requirement Spec

YAML file defining VLANs, subnets, gateways & security rules

2

Assurance Tool

smartbranch_checker.py — automated config verification

3

Design Document

Consolidated topology, VLAN/IP table & security notes

4

Fault Cards + Demo

5 fault scenarios documented + 5–10 min demo video

3.1 — Requirement Specification (YAML)

Before Members 1 & 2 began configuration, I authored a machine-readable requirement file so the network design, and later the assurance tool, share a single source of truth for VLANs, subnets, gateways and security rules.

```
vlan:
- id: 10
  name: EMPLOYEE
  network: 10.10.10.0/24
  gateway: 10.10.10.1
- id: 20
  name: GUEST
  network: 10.10.20.0/24
  gateway: 10.10.20.1

security:
  guest_to_server: deny
  guest_to_management: deny

nat:
  enabled: true
  overload: true
```

VLAN / IP DESIGN TABLE

VLAN	Name	Network	Gateway
10	EMPLOYEE	10.10.10.0/24	10.10.10.1
20	GUEST	10.10.20.0/24	10.10.20.1
30	SERVER	10.10.30.0/24	10.10.30.1
99	MANAGEMENT	10.10.99.0/24	10.10.99.1

Why this matters

This file drives both the design document and the checker script — Members 1 & 2 build to it, and the tool tests against it, so drift between design and configuration is caught automatically.

3.2 — Python Assurance Tool (smartbranch_checker.py)

I wrote a Python script that parses the router and switch CLI output captured from the group's Packet Tracer file, and checks it against the VLAN, gateway, DHCP, security, NAT and SSH values defined in the requirement spec.

```
def check_security(config):
    passed = 0
    guest_server = (
        "deny ip 10.10.20.0 0.0.0.255 "
        "10.10.30.0 0.0.0.255"
    )
    if guest_server in config:
        print("[PASS] Guest -> Server blocked")
        passed += 1
    ...
    return passed
```

WHAT IT CHECKS

- VLAN presence & naming (4 VLANs)
- Trunk allowed-VLAN list on Fa0/1
- Sub-interface gateway addresses
- DHCP pool network & default-router
- Guest ACL deny rules
- NAT overload configuration
- SSH version 2 & management restriction

3.3 — Running the Assurance Tool

The script reads the exported router and switch CLI configuration and prints a PASS/FAIL line for every requirement, followed by a final score.

```
=====
SMART BRANCH NETWORK ASSURANCE CHECKER
=====

--- SECURITY CHECK ---
[PASS] Guest -> Server blocked
[PASS] Guest -> Management blocked

--- NAT CHECK ---
[PASS] NAT overload configured

--- SSH CHECK ---
[PASS] SSH version 2 enabled
[PASS] SSH restricted to Management VLAN

RESULT: 21/21 checks passed
STATUS: ALL CHECKS PASSED
```

21/21

checks passed on final run

Covers VLANs, trunk state, gateways, DHCP pools, Guest ACL, NAT overload and SSH hardening — every requirement in the YAML spec, verified in one run.

3.4 — Compiling the Design Document

I assembled the group's individual outputs — Member 1's topology and addressing, Member 2's security and NAT configuration — into a single design document covering the network end to end.

Topology Diagram

Annotated logical diagram of the shared Packet Tracer file — routers, switches, servers, AP and endpoints across 4 VLANs.

VLAN / IP Table

Consolidated addressing plan: VLAN ID, name, network and gateway for Employee, Guest, Server and Management.

Security Notes

Summary of the Guest ACL, SSH-to-Management restriction and NAT overload policy, with match-count evidence.

Verification Record

The assurance tool's 21/21 result and the end-to-end test outcomes, included as design proof.

3.5 — Fault Card Documentation

Member 2 injected and diagnosed five faults during testing; I documented each as a fault card — symptom, root cause and fix — for the design document.

Fault	Symptom	Root Cause	Fix
Wrong default gateway	PCs get a DHCP lease but can't reach any other VLAN or the Internet	Sub-interface IP mistyped on Gi0/0.x	Correct the ip address on the sub-interface; verify with show ip interface brief
Guest VLAN removed from trunk	Guest devices lose DHCP and drop off the network entirely	switchport trunk allowed vlan remove 20 run on uplink	Re-add with allowed vlan add 20; verify via show interfaces trunk
Incorrect DHCP network	Employee PCs get no lease or an address outside the expected subnet	DHCP pool network statement pointed to wrong subnet	Correct the network statement; release/renew and reverify
DNS blocked by ACL	Hosts reach the Internet by IP but names fail to resolve	GUEST-RESTRICT ACL missing a permit for port 53	Insert explicit DNS permit before the deny lines; retest nslookup
NAT failure	Internal hosts ping the router but no external address	ip nat inside/outside missing, or source list 1 misconfigured	Reapply NAT on correct interfaces; verify with show ip nat translations

3.6 — Demo Video Recording

I recorded a 5–10 minute walkthrough of the finished network, following the demo flow used for evaluation: build, security, testing, and assurance.

- 1 Introduce the topology** Walk through the logical diagram — routers, switches, VLANs and endpoints.
- 2 Show connectivity** Demonstrate DHCP leases, inter-VLAN routing and Internet access via NAT.
- 3 Demonstrate security** Show Guest ACL blocking Server/Management traffic and SSH restricted to Management VLAN.
- 4 Inject a fault live** Break one configuration item, show the symptom, then apply the documented fix.
- 5 Run the assurance tool** Execute `smartbranch_checker.py` and show the 21/21 result confirming the fix.

Every Requirement Verified by the Tool

21/21

Python assurance tool
checks passed

- ✓ VLAN 10/20/30/99 present & named correctly
- ✓ All 4 VLANs allowed on the trunk
- ✓ Sub-interface gateways match the design
- ✓ DHCP pools bound to correct network & router
- ✓ Guest ACL blocks Guest → Server traffic
- ✓ Guest ACL blocks Guest → Management traffic
- ✓ NAT overload configured on outside interface
- ✓ SSH v2 enabled & restricted to Management VLAN

Skills and Learning Outcomes

✓ Translating a network design into a structured YAML requirement spec

✓ Writing Python (regex-based) parsers for Cisco IOS CLI output

✓ Building an automated configuration-assurance tool from scratch

✓ Reading and interpreting show vlan / show interfaces trunk / running-config

✓ Compiling multi-contributor technical work into one design document

✓ Documenting fault scenarios as symptom / root cause / fix cards

✓ Planning and recording a structured technical demo video

✓ Collaborating in a 3-member team on a shared Packet Tracer topology

CONCLUSION

Python Tool, Documentation & Demo — Delivered and Verified

As Member 3, I authored the YAML requirement specification, built the `smartbranch_checker.py` assurance tool, compiled the group's design document from Members 1 and 2's work, documented all five fault-injection cards, and recorded the demo video. The tool's 21/21 result — covering VLANs, gateways, DHCP, Guest ACL, NAT and SSH — gives the whole project a repeatable, automated proof that the shared configuration matches the design.

Thank you

Akhil Gwari • B.Tech CSE, IILM University • 2410030441